

Lab 4: Parameterized and Data-Driven Tests

Objective

Learn how to use NUnit's parameterized and data-driven test features to test multiple scenarios efficiently.

Prerequisites

- Completion of **Lab 3** or familiarity with mocking and dependency isolation.
- Basic understanding of NUnit test attributes like `[Test]`.

Instructions

Step 1: Add a Discount Calculator Class

1. In the `NunitLab.Api` project, create a new class `DiscountCalculator.cs`:

```
namespace NunitLab.Api;

public class DiscountCalculator
{
    public decimal CalculateDiscount(decimal totalAmount, string membershipType)
    {
        return membershipType switch
        {
            "Regular" => totalAmount * 0.05m,
            "Premium" => totalAmount * 0.10m,
            "VIP" => totalAmount * 0.20m,
            _ => 0m
        };
    }
}
```

```
DiscountCalculator.cs
NunitLab.Api
1 namespace NunitLab.Api;
2
3 public class DiscountCalculator
4 {
5     public decimal CalculateDiscount(decimal totalAmount, string membershipType)
6     {
7         return membershipType switch
8         {
9             "Regular" => totalAmount * 0.05m,
10            "Premium" => totalAmount * 0.10m,
11            "VIP" => totalAmount * 0.20m,
12            _ => 0m
13        };
14    }
15 }
16
```

Step 2: Write Parameterized Tests

1. In the `NunitLab.UnitTests` project, create a new test class `DiscountCalculatorTests.cs`:

```
using NunitLab.Api;

namespace NunitLab.UnitTests;

[TestFixture]
public class DiscountCalculatorTests
{
    private DiscountCalculator? _systemUnderTest;

    [SetUp]
    public void Setup()
    {
        _systemUnderTest = null;
    }

    public DiscountCalculator SystemUnderTest
    {
        get
        {
            if (_systemUnderTest == null)
            {
                _systemUnderTest = new DiscountCalculator();
            }

            Assert.That(_systemUnderTest, Is.Not.Null);
        }
    }
}
```

```
        return _systemUnderTest;
    }
}

[TestCase(100, "Regular", 5)]
[TestCase(200, "Premium", 20)]
[TestCase(300, "VIP", 60)]
[TestCase(150, "Unknown", 0)]
public void CalculateDiscount_ShouldReturnExpectedDiscount(
    decimal totalAmount, string membershipType, decimal expectedDiscount)
{
    // Act
    decimal actualDiscount = SystemUnderTest.CalculateDiscount(totalAmount,
membershipType);

    // Assert
    Assert.That(actualDiscount, Is.EqualTo(expectedDiscount));
}
}
```

```
DiscountCalculatorTests.cs
NunitLab.UnitTests
NunitLab.UnitTests.DiscountCalculatorTests

1  using NunitLab.Api;
2
3  namespace NunitLab.UnitTests;
4
5  [TestFixture]
6  public class DiscountCalculatorTests
7  {
8      private DiscountCalculator? _systemUnderTest;
9
10     [SetUp]
11     public void Setup()
12     {
13         _systemUnderTest = null;
14     }
15
16     public DiscountCalculator SystemUnderTest
17     {
18         get
19         {
20             if (_systemUnderTest == null)
21             {
22                 _systemUnderTest = new DiscountCalculator();
23             }
24
25             Assert.That(_systemUnderTest, Is.Not.Null);
26
27             return _systemUnderTest;
28         }
29     }
30
31     [TestCase(100, "Regular", 5)]
32     [TestCase(200, "Premium", 20)]
33     [TestCase(300, "VIP", 60)]
34     [TestCase(150, "Unknown", 0)]
35     public void CalculateDiscount_ShouldReturnExpectedDiscount(
36         decimal totalAmount, string membershipType, decimal expectedDiscount)
37     {
38         // Act
39         decimal actualDiscount = SystemUnderTest.CalculateDiscount(totalAmount, membershipType);
40
41         // Assert
42         Assert.That(actualDiscount, Is.EqualTo(expectedDiscount));
43     }
44 }
45
```

Step 3: Use [TestCaseSource] for Complex Data Sets

1. In **DiscountCalculatorTests.cs**, add a method to supply test data:

```
public static IEnumerable<object[]> DiscountTestCases()
{
    yield return new object[] { 500m, "Regular", 25m };
    yield return new object[] { 400m, "Premium", 40m };
    yield return new object[] { 1000m, "VIP", 200m };
}
```

2. Use `[TestCaseSource]` to reference the data source:

```
[Test, TestCaseSource(nameof(DiscountTestCases))]
public void CalculateDiscount_WithTestCaseSource_ShouldReturnExpectedDiscount(
    decimal totalAmount, string membershipType, decimal expectedDiscount)
{
    // Act
    decimal actualDiscount = SystemUnderTest.CalculateDiscount(
        totalAmount, membershipType);

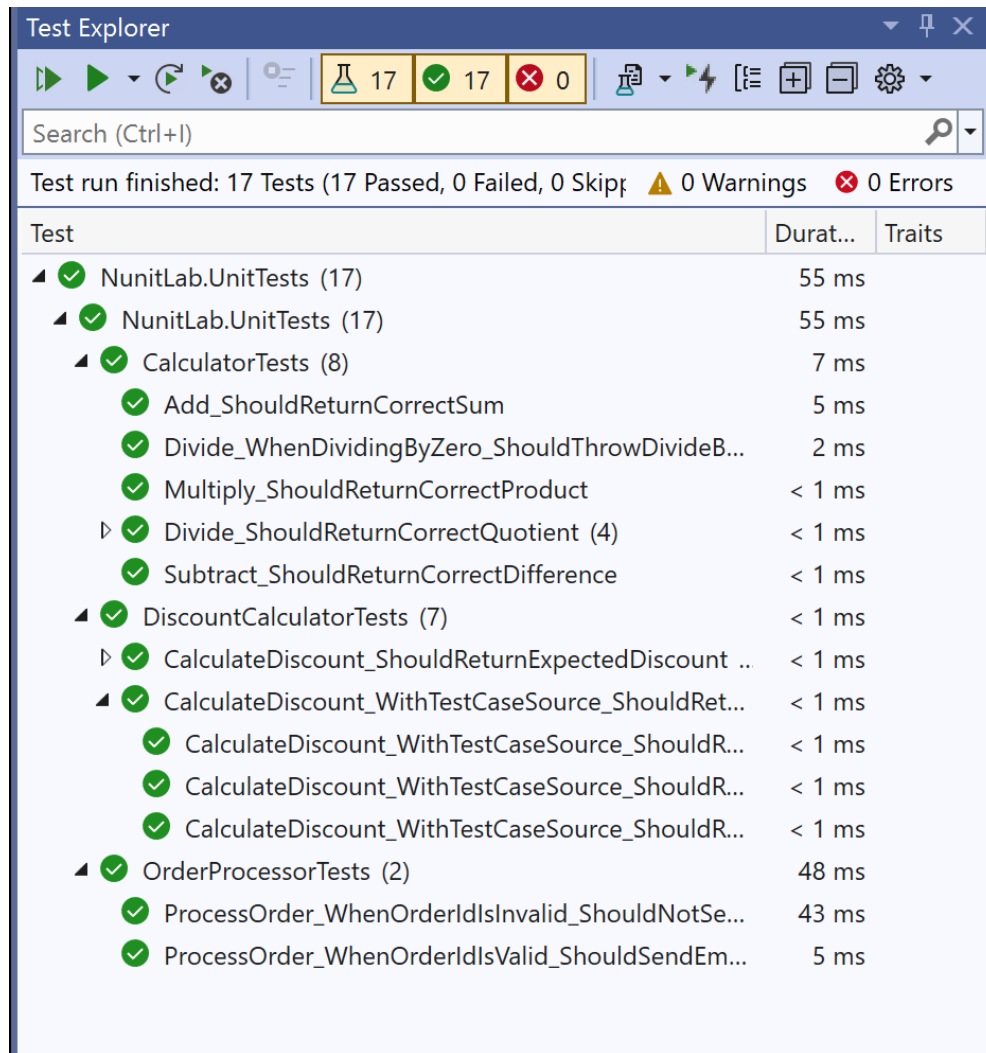
    // Assert
    Assert.That(actualDiscount, Is.EqualTo(expectedDiscount));
}
```

```
DiscountCalculatorTests.cs
NunitLab.UnitTests
NunitLab.UnitTests.DiscountCalculatorTests

6 public class DiscountCalculatorTests
16     public DiscountCalculator SystemUnderTest
18     get
28     {
29     }
30
31     [TestCase(100, "Regular", 5)]
32     [TestCase(200, "Premium", 20)]
33     [TestCase(300, "VIP", 60)]
34     [TestCase(150, "Unknown", 0)]
35     public void CalculateDiscount_ShouldReturnExpectedDiscount(
36         decimal totalAmount, string membershipType, decimal expectedDiscount)
37     {
38         // Act
39         decimal actualDiscount = SystemUnderTest.CalculateDiscount(totalAmount, membershipType);
40
41         // Assert
42         Assert.That(actualDiscount, Is.EqualTo(expectedDiscount));
43     }
44
45     public static IEnumerable<object[]> DiscountTestCases()
46     {
47         yield return new object[] { 500m, "Regular", 25m };
48         yield return new object[] { 400m, "Premium", 40m };
49         yield return new object[] { 1000m, "VIP", 200m };
50     }
51
52     [Test, TestCaseSource(nameof(DiscountTestCases))]
53     public void CalculateDiscount_WithTestCaseSource_ShouldReturnExpectedDiscount(
54         decimal totalAmount, string membershipType, decimal expectedDiscount)
55     {
56         // Act
57         decimal actualDiscount = SystemUnderTest.CalculateDiscount(
58             totalAmount, membershipType);
59
60         // Assert
61         Assert.That(actualDiscount, Is.EqualTo(expectedDiscount));
62     }
63
64 }
```

Step 4: Run the Tests

1. Open the **Test Explorer** in Visual Studio.
2. Run all tests and verify that:
 - All parameterized test cases pass.
 - Tests using `[TestCaseSource]` execute as expected.



Outcome

Students will:

- Understand how to use `[TestCase]` for parameterized tests.
- Leverage `[TestCaseSource]` for complex or dynamically generated test data.